AIClubPro

# Introduction to Agents

# Overview

- Agentic AI is everywhere - including in Google Gemini/NotebookLM, OpenAI Codex, and Anthropic Claude Code/ Cowork
- There are two dominant styles: Workflow and Autonomous.
- Focus today is on Workflow Agents - we will cover Autonomous Agents later.

# Lets go back to 2024

- ChatGPT is really popular
- APIs are now well-established
- You want to use AI for customer support.

# The first attempt: Big monolith

## Create a single grand prompt to cover all use cases.

- Customer Support chatbot: status requests ("Where is my product?")
- complaint ("Why did the product not work?")
- happy customer (rare - can we getting testimonials?).
- polite chatter, and refer user to human agents if needed.

# Monolith structure

- Header (including "be polite", being on-brand)
- Multiple policy sections - one for each use case
- Tooling (in case you need to check shipping system etc.)

# The good of the monolith

- Simple to set up
- Generally fast
- Works well for simple requests

# Challenges with monoliths

- LLMs had limited context windows (typical 8,000 tokens - includes all input and output combined)
- LLMs tended to make more mistakes when they were given lots of directions.
- Hard to maintain and reason through - what happens when the company adds the 5th use case (or 10th)?
- Or what if the AI Agent has to carry out several actions to address the user's input.

# Solution: Modular Engineering

- Divide-and-conquer
- Follow a multi-step approach to solve problems
- Multiple specialists vs. one generalist

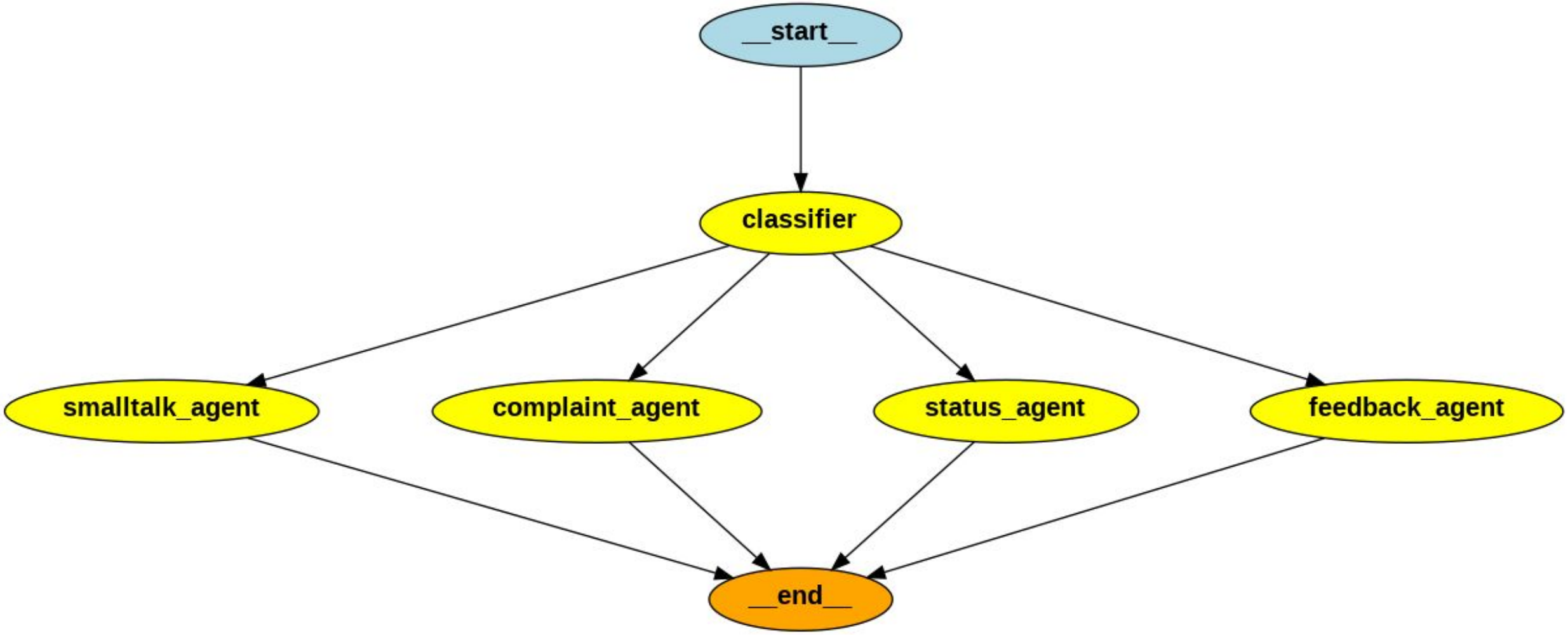
# Solution: Workflow Agents as Modular Engineering

- Each Agent does one job really well
- "Front Desk" Agent performs triage; hands over to specialist Agents
- Workflow system manages flow of control and information between Agents.

# Benefits of this architecture

- Can update each independently
- If there are issues, we can track and figure out which Agent/prompt has an issue
- Can add more specialists without impacting most of the system

# A Basic Workflow Agentic AI system



# What is an Agent?

## Key Characteristics

An Agent is a piece of code or method.

Typically has its own dedicated Prompt.

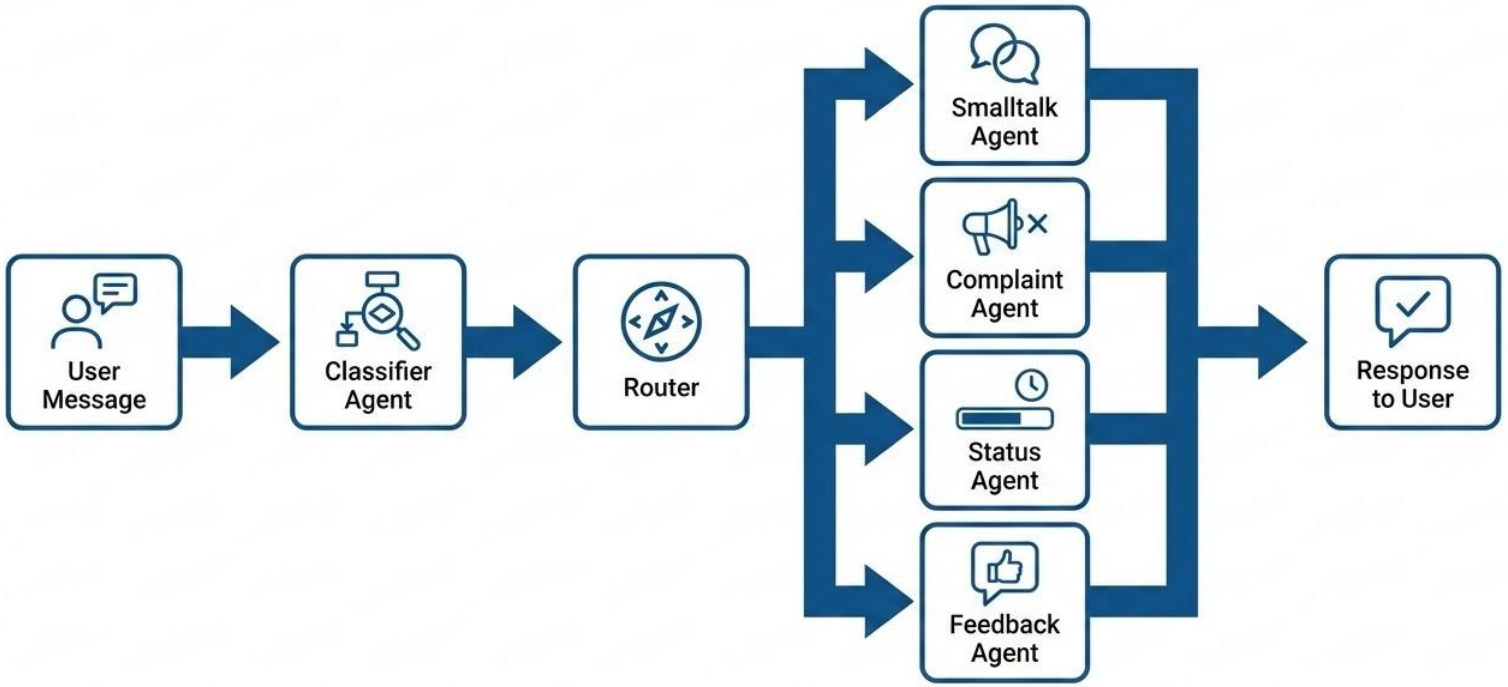
Carries out one particular processing task.

## Sample Agent Code

```
def complaint_agent(self, state: AgentState):
    print("Complaint agent processing....")
    COMPLAINT_PROMPT = f"""
    You are a complaint agent that addresses
    user complaints.
    Given the following messages, respond
    appropriately to the user's complaint.
    """
    llm_messages = create_llm_msg(
    COMPLAINT_PROMPT, state.messages)
    return {"response":
    self.model.stream(llm_messages),
        "category": "complaint_agent"}
```

# In this example, we have five Agents

- Classifier Agent
- Smalltalk Agent
- Complaint Agent
- Status Agent
- Feedback Agent




- How do Agents communicate with each other?

# How do Agents communicate with each other?

- This is specific to choices made in Langgraph
- You could do things differently
- This is a good design

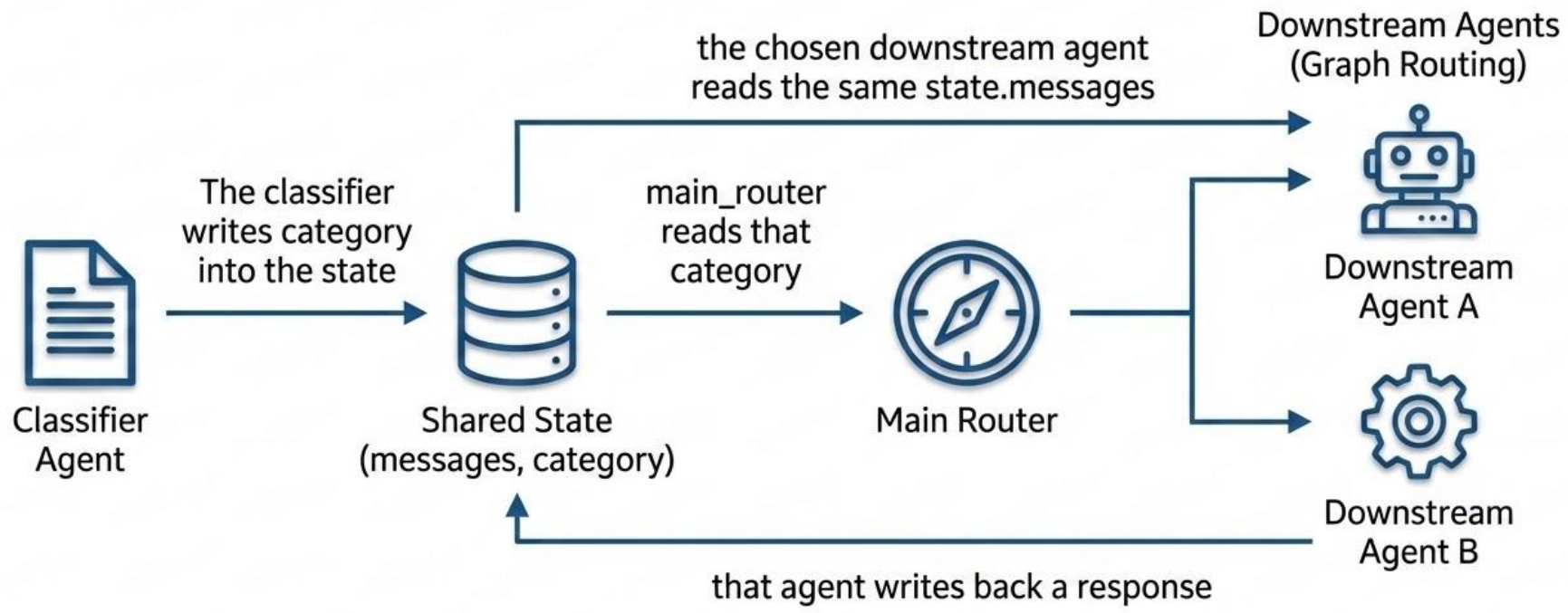
# Designing the communication layer

- You have a few Agents
- Each Agent is a Python function
- The function can accept arbitrary parameters
- How does the infrastructure know the parameters for each function and pass the corresponding information?

- Solution: One single shared “state” object
- Every function has the same signature
- The object is updated by each function as we progress

# How do agents communicate with each other?

## Indirect Communication via Shared State & Graph Routing




```python
class AgentState(BaseModel):
    """State of the agent."""
    messages: list = []
    response: str = ""
    category: str = ""

class Category(BaseModel):
    """Category for the agent."""
    category: str
```

So it is not agent-to-agent chat. It is more like:
**state-based coordination inside an orchestration graph.**

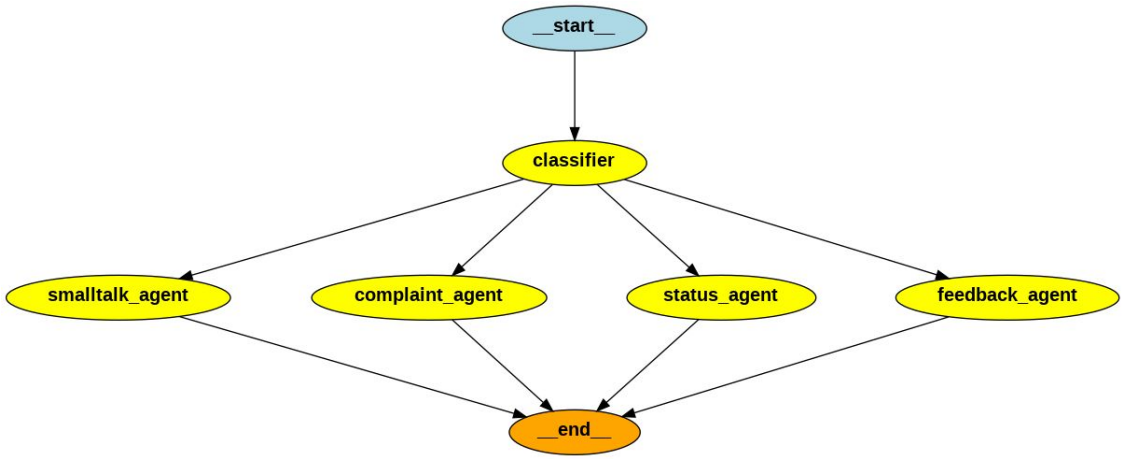
# Control flow between Agents

```
workflow = StateGraph(AgentState)

 workflow.add_node("classifier", self.classifier)
 workflow.add_node("smalltalk_agent", self.smalltalk_agent)
 workflow.add_node("complaint_agent", self.complaint_agent)
 workflow.add_node("status_agent", self.status_agent)
 workflow.add_node("feedback_agent", self.feedback_agent)

 workflow.add_edge(START, "classifier")
 workflow.add_conditional_edges("classifier", self.main_router)
 workflow.add_edge("smalltalk_agent", END)
 workflow.add_edge("complaint_agent", END)
 workflow.add_edge("status_agent", END)
 workflow.add_edge("feedback_agent", END)

 self.graph = workflow.compile()
```




- Service provided by Langgraph library

# AgentState manages shared state between Agents

```
class AgentState(BaseModel):
    """State of the agent."""
    messages: list = []
    response: str = ""
    category: str = ""
```

- Service provided by Langgraph library

# Tool Usage

- We “brute-forced” it in Status Agent
- Usually the tool calls are conditional
- Built-in support in the LLM APIs

# Thank you!

**Connect with us.**

info@aiclub.world

www.aiclubpro.world

aiclubpro